

仕様書作成と確認



Thought for 50s >

結論から言うと、この「ポートフォリオ」ページはコンセプトは良いけど、今の形だとかなり中途半端です。
このまま出すより、「何を決めるページなのか」を絞って作り直した方がいい。

1. いまのポートフォリオ画面の評価

良い点 (コンセプトとしてアリなところ)

- 「Multi-Objective Pareto Frontier」で
利益 vs リスク vs CAS をまとめて見る、という発想自体は世界水準に近い。
- 上部の「Recommended Portfolio Strategy」カードは
 - 期待Δ%
 - CAS
 - Risk
 - ROIをまとめて見せていて、経営層向けの“意思決定カード”としては筋が良い。

→ なので「ページごとボツ」ではなく、**役割を明確化してスリムにする方向**が良いと思う。

問題点 (正直ここがキツい)

- ポリシーA〜Qが意味不明
 - ユーザが明示的に作っていないのに「Policy A〜Q」がズラッと出てくると、
 - 「これ何を意味しているの？」
 - 「どこから出てきた数字？」となって信用が一気に落ちる。
 - サンプルデータだとしても、**本番UIで“架空の施策”を大量表示するのはNG**。
- リストが長いだけで“読み方”が分からない
 - 全部似た見た目のカードが縦に並んでいて、
 - どれが推奨？
 - どれがフロンティア上？
 - どれを捨てろと言っている？が一目で分からない。
 - 「Policy A - Detailed Specifications」も
経営者視点でいうと「で、これを打つべきか・やめるべきか」の一言が弱い。
- ポートフォリオというより“**ただの施策一覧 2枚目**”になっている
 - Policy Lab でも施策は見えているので、
同じ情報をレイアウトだけ変えて二重に見せている印象になりがち。
 - 「このページでしか得られない示唆」が弱い。
- Pareto Frontier の活かし方が弱い**
 - グラフ自体は良いが、
 - 推奨ポートフォリオの点がどれか
 - 他の候補とどう違うのか
 - リスク制約を変えるとどう動くのかが分からない。
 - ただの「散布図」に見えてしまうリスクが高い。

2. どう変えるのが良いか (遠慮なし案)

役割を一言で定義し直す

「複数の候補施策から、どの組み合わせで予算を張るかを決めるページ」

に徹した方がいいです。

「マーケ横断の投資委員会で使うスライド1枚分」をUIにしたイメージ。

構成案

1. 上部：推奨ポートフォリオの“結論カード”だけ残す

- すでにあるこのブロックはかなり良いので、ここを主役にする：
 - Expected $\Delta\yen$ (円表記も併記)
 - Total Budget / Expected Cost
 - Portfolio Risk (CVaR または %)
 - Mean CAS
- ここに
 - 「選択されたポリシー数：5 / 17」
 - 「主なリスク要因：Channel = Paid Social, Segment = Young」
くらいを一行で出すと“それっぽさ”が一気に上がる。

2. 中央：Pareto Frontier + ポートフォリオ構成

- 散布図は残すが、役割を明確にする：
 - ハイライト：推奨ポートフォリオ (大きめの点 + ラベル)
 - 他：候補ポートフォリオ (薄い点)
- 右側か下に
「ポートフォリオ構成 (寄与度ランキング)」の簡易バー：
 - Policy X: +¥80K (60%)
 - Policy Y: +¥30K (25%)
 - Policy Z: +¥20K (15%)
- ここで初めて**「どの施策がこの結論を支えているか」**が分かる。

3. 下部：テーブルは“推奨/非推奨”だけにシンプル化

- 今のA~Qの長いカードはやめて、
1行 = 1 policy のテーブルで十分：
 - Policy Name
 - $\Delta\yen$
 - Risk
 - ROI
 - CAS
 - Verdict (Include / Exclude / Test)
- 「Include」だけ色を付けて、“ポートフォリオ採用施策”が一目で分かるように。

4. プレースホルダは絶対やめる

- ユーザが Policy Lab で1件も施策を作っていないなら：
 - 「ポートフォリオ最適化を行うには、まずポリシーラボで2件以上の施策を評価してください」
のような空状態 (empty state) だけを出す。
- サンプルデータを触らせたい時は、「サンプルポリシーを読み込む」ボタンで明示した方が良い。

3. このページを残すかどうかについて

特にポリシー設定していないのに、Qまであるし。。。。

この違和感は完全に正しくて、現状のままなら「無い方がマシ」に近いです。

ただし：

- 「単一施策の $\Delta\yen$ だけではなく、複数施策をどう組み合わせるか」は
Cレベルの意思決定で必ず聞かれるポイント。
- そこまで踏み込んだダッシュボードを持っている競合はほとんどいない。

ので、

- ページは残す方針でOK
- ただし
 - プレースホルダ施策は全削除
 - 構成を「推奨ポートフォリオ結論 + フロンティア + シンプルな採用一覧」に縮約

- Policy Lab と役割がかぶる部分は徹底的に削る

という方向が良いと考えます。

もしよければ次のステップとして：

1. このページの“理想ワイヤーフレーム”をテキストで一緒に固める
2. それを Claude 用の仕様 + React/TanStack Query 用のコンポーネント構成に落とす

までセットでやりましょう。

そうすれば「Portfolio は世界級、Decision Console は経営スナップショット」というきれいな役割分担にできます。



Thought for 25s >

以下、そのまま Claude に渡せる形で書きます。

(React + TypeScript + TanStack Query + FastAPI 前提で記述していますが、必要に応じて読み替え前提で OK です)

0. ゴール概要 (Claude 向け要約)

やりたいこと

既存の「Marketing Portfolio & ROI (/portfolio)」ページを作り直す。

役割は「複数の施策 (Policy) の組み合わせ = ポートフォリオ」を評価・推奨する C レベル向け画面。

ただの施策一覧ではなく、「ポートフォリオとして何を採用すべきか」を一目で分かる UI にする。

1. 現状と問題点 (前提共有)

現状の問題

- ユーザが明示的に作っていないのに Policy A~Q のような「ダミー施策」が大量に並んでいる
→ 何がどこから来た数字か分からず、信用を落とすリスクが高い。
- Policy Lab と似た内容のリストをもう一度見ているだけに近く、「このページでしか得られない意思決定」が弱い。
- Pareto Frontier はあるが、
 - 推奨ポートフォリオの点がどれか
 - 他候補とどう違うのか
 - リスク制約を変えるとどう動くかが分からない。
- 空状態の UX が弱い (施策ゼロでもダミー表示されるなど)。

新デザインのコンセプト

「複数の候補施策から、どの組み合わせで予算を張るかを決めるための画面」

- 想定ユーザ：マーケティング責任者 / 経営層
- 1 秒で理解したい情報：
 - この期間、どのポートフォリオ構成が推奨か
 - そのときの Δ / ROI / リスク / 信頼度 (CAS)
 - その結果を支えている主要施策はどれか

2. 新 UI の構成 (最終形イメージ)

ページタイトル・ルーティング

- タイトル：Marketing Portfolio & ROI
- パス：/portfolio
- サイドバー：名称はそのまま「ポートフォリオ」

レイアウト構成 (上から順)

1. ヘッダー (シナリオ情報)

- Scenario 名 (例: Multi-Policy Portfolio Optimization)
- Target metric (例: $y = \text{Incremental Profit vs Risk}$)
- 期間フィルタ (ボタン 3 つ)
 - Last 14 days / Last 28 days / Last 56 days
 - クリックでページ全体のデータを再読込

2. 結論カード: Recommended Portfolio Strategy (主役)

- Expected Δy (円表示)
- Total Budget / Estimated Cost (わかる範囲で)
- Portfolio Risk Score (0~1)
- Mean CAS Score
- 簡単なテキスト説明:
 - 「Policy X, Y, Z を含むポートフォリオが、 $CAS \geq 0.8$ & $Risk \leq 0.3$ の制約下で最も高い Δy を提供します」など

3. 中段: 可視化エリア

3-1. Pareto Frontier (Profit vs Risk)

- x 軸: Risk Score
- y 軸: Δy (円)
- プロット:
 - 推奨ポートフォリオ: 大きな点 + ラベル Recommended
 - 他の候補ポートフォリオ: 小さな点
- 凡例:
 - 色 or マーカー形状 = CAS のレンジ (High / Medium / Low)

3-2. Portfolio Contribution (寄与度ランキング)

- サイド or 下にバークラフ:
 - x 軸: 各 Policy
 - y 軸: そのポートフォリオ全体 Δy に対する寄与額
- 上位 5 件だけ表示 + 「View all policies」リンクでテーブルへスクロール

4. 下段: Portfolio Policies Table

- テーブル列:
 - Policy Name
 - Dataset
 - Channel
 - Segment
 - Δy (円)
 - ROI (x)
 - Risk
 - CAS
 - Verdict (Include / Exclude / Test など)
- フィルタ:
 - Verdict (All / Include / Test / Exclude)
- 行クリック:
 - 該当 Policy を右側パネル (詳細) で開く or Modal で詳細表示 (既存の「Policy Detailed Specifications」を再利用)

5. Empty state / エラー時

- ポリシー 0 件:
 - 「ポートフォリオ最適化を行うには、Policy Lab で 2 件以上の施策を評価してください」
 - 「サンプルポリシーを読み込む」ボタン (デモモードのみ)
- 計算失敗:
 - 「ポートフォリオ最適化が実行できませんでした。施策数が 1 件以下の可能性があります」など、原因と次アクションを明示。

3. データモデルと API 仕様 (提案)

既存 API を壊さない範囲で、できるだけ追加・ラップする形を想定。

3.1 TypeScript 型 (フロント側)

```
export type PortfolioWindow = "14d" | "28d" | "56d";

export interface PortfolioSummary {
  window: PortfolioWindow;
  expectedDeltaYen: number; // 推奨ポートフォリオの期待Δ¥
  totalCostYen?: number; // 任意
  meanCas: number; // 0-1
  portfolioRisk: number; // 0-1
  roi: number; // x 倍 (例: 5 -> 5x)
  selectedPolicyIds: string[]; // 採用されている Policy IDs
}

export interface PortfolioPolicy {
  id: string;
  name: string;
  datasetName: string;
  channel?: string;
  segment?: string;
  deltaYen: number;
  roi: number;
  risk: number;
  cas: number;
  verdict: "include" | "exclude" | "test";
}

export interface ParetoPoint {
  portfolioId: string;
  risk: number;
  deltaYen: number;
  meanCas: number;
  isRecommended: boolean;
}
```

What this does (このコードの説明)

- ポートフォリオ画面で使う最小限のドメイン型を定義している。
- `PortfolioSummary` は「推奨ポートフォリオ全体」の集計値。
- `PortfolioPolicy` はそのポートフォリオに含まれ得る個々の施策 (Policy) の指標。
- `ParetoPoint` はフロンティア散布図の 1 点 (1 つの候補ポートフォリオ)。
- 期間は `PortfolioWindow` で 14/28/56 日を指定する前提。

Expert insight (Google/Meta/NASA level)

型は「ページ専用 DTO」として切り出して、バックエンドの生のレスポンスに直接依存しない形にしておく、将来のモデル変更 (リスク指標を CVaR→VaR に変えるなど) を UI から切り離せる。

3.2 REST API (バックエンド案)

既存のポートフォリオ最適化エンドポイントをラップするイメージで OK。

1. ポートフォリオ概要

```
GET /api/portfolio/summary?window=28d
```

レスポンス例：

```
{
  "window": "28d",
  "expectedDeltaYen": 133177,
  "totalCostYen": 26635,
  "meanCas": 0.75,
  "portfolioRisk": 0.15,
  "roi": 5.0,
  "selectedPolicyIds": ["pol-001", "pol-003", "pol-004"]
}
```

2. ポートフォリオ候補一覧 (テーブル用)

```
GET /api/portfolio/policies?window=28d
```

レスポンス例：

```
[
  {
    "id": "pol-001",
    "name": "Dynamic Discount Policy",
    "datasetName": "marketing_campaign_10k_processed",
    "channel": "email",
    "segment": "high_value",
    "deltaYen": 80000,
    "roi": 4.0,
    "risk": 0.12,
    "cas": 0.82,
    "verdict": "include"
  }
  // ...
]
```

3. Pareto Frontier

GET /api/portfolio/frontier?window=28d

レスポンス例：

```
[
  {
    "portfolioId": "pf-001",
    "risk": 0.10,
    "deltaYen": 120000,
    "meanCas": 0.8,
    "isRecommended": true
  },
  {
    "portfolioId": "pf-002",
    "risk": 0.05,
    "deltaYen": 90000,
    "meanCas": 0.85,
    "isRecommended": false
  }
]
```

What this does

- フロントが必要とする 3 種類の情報（サマリ、施策一覧、フロンティア点）を、window（14/28/56 日）でフィルタして取得できるようにしている。
- バックエンド内部の最適化ロジックを変えても、上記スキーマを守ればフロントは変更不要。

Expert insight

ポートフォリオの推奨は計算コストが高くなりがちなので、**window + データセット構成で結果をキャッシュ**しておく設計にしておくと、C レベルの閲覧時でもレスポンスが安定する。

4. React コンポーネント設計

コンポーネント構造

```
PortfolioPage
├─ PortfolioHeader           // 期間フィルタ + Scenario 情報
├─ PortfolioSummaryCard     // Recommended Portfolio Strategy カード
├─ PortfolioVisualizationRow
│   ├─ ParetoFrontierChart  // 散布図
│   └─ PortfolioContribution // 寄与度バークラフ
└─ PortfolioPoliciesTable   // 施策テーブル + Verdict フィルタ
```

データ取得 (TanStack Query)

```
import { useQuery } from "@tanstack/react-query";

function usePortfolioData(window: PortfolioWindow) {
  const summaryQuery = useQuery({
```

```

queryKey: ["portfolio-summary", window],
queryFn: async (): Promise<PortfolioSummary> => {
  const res = await fetch(`/api/portfolio/summary?window=${window}`);
  if (!res.ok) throw new Error("Failed to fetch portfolio summary");
  return res.json();
},
});

const policiesQuery = useQuery({
  queryKey: ["portfolio-policies", window],
  queryFn: async (): Promise<PortfolioPolicy[]> => {
    const res = await fetch(`/api/portfolio/policies?window=${window}`);
    if (!res.ok) throw new Error("Failed to fetch portfolio policies");
    return res.json();
  },
});

const frontierQuery = useQuery({
  queryKey: ["portfolio-frontier", window],
  queryFn: async (): Promise<ParetoPoint[]> => {
    const res = await fetch(`/api/portfolio/frontier?window=${window}`);
    if (!res.ok) throw new Error("Failed to fetch frontier points");
    return res.json();
  },
});

return { summaryQuery, policiesQuery, frontierQuery };
}

```

What this does

- 指定された `window` (14/28/56 日) に対して、サマリ・施策一覧・フロンティア点を並列で取得するカスタムフック。
- React Query の `queryKey` に `window` を含めることで、期間を切り替えたときにキャッシュの切り替えと再フェッチが自動で行われる。

Expert insight

複数クエリをまとめるカスタムフックを作るときは、**「ローディング状態の統合の仕方」**を明示的に決めておくとうい。例えば `PortfolioPage` 側では `isLoading = summaryQuery.isLoading || policiesQuery.isLoading || frontierQuery.isLoading` のように揃えておくスケルトン表示の仕様がブレない。

PortfolioPage スケルトン

```

export const PortfolioPage: React.FC = () => {
  const [window, setWindow] = useState<PortfolioWindow>("28d");
  const { summaryQuery, policiesQuery, frontierQuery } = usePortfolioData(window);

  const isLoading = summaryQuery.isLoading || policiesQuery.isLoading || frontierQuery.isLoading;
  const hasError = summaryQuery.isError || policiesQuery.isError || frontierQuery.isError;

  if (isLoading) {
    // TODO: スケルトン UI
    return <div>Loading portfolio...</div>;
  }

  if (hasError) {
    return (
      <div>
        ポートフォリオ情報の取得に失敗しました。
        施策数が 1 件以下、もしくはバックエンドで最適化が失敗している可能性があります。
      </div>
    );
  }

  const summary = summaryQuery.data!;
  const policies = policiesQuery.data ?? [];
  const frontier = frontierQuery.data ?? [];

  if (policies.length === 0) {
    return (

```

```

    <EmptyPortfolioState window={window} onChangeWindow={setWindow} />
  );
}

return (
  <div className="flex flex-col gap-6">
    <PortfolioHeader window={window} onChangeWindow={setWindow} />
    <PortfolioSummaryCard summary={summary} />
    <PortfolioVisualizationRow frontier={frontier} policies={policies} summary={summary} />
    <PortfolioPoliciesTable policies={policies} summary={summary} />
  </div>
);
};

```

What this does

- 期間選択 state (`window`) を持つメインコンポーネント。
- 3 つの Query のローディング・エラー状態を統合し、
 - Loading スケルトン
 - エラー文言
 - ポリシー 0 件の空状態
 の 3 パターンを分けている。
- データがある場合は、上から順にヘッダー → サマリカード → 可視化 → テーブルを描画する。

Expert insight

プロダクトの“キーページ”では、ローディング・エラー・空状態を**「if で早期 return」**する書き方にしておくと、メインの正常系 JSX が読みやすくなり、将来の AB テストやレイアウト変更時に事故が減る。

5. ロジック・メトリクスの定義

実装時にあいまいになりそうなところを明示しておく。

1. **Expected Δ¥**
 - ポートフォリオに含まれる Policy の Δ¥ の合計
 - ただし重複ユーザをどう扱うかはバックエンドの責務
 - フロントは「バックエンドから渡された Δ¥ を円表記で出すだけ」
2. **Portfolio Risk**
 - 0～1 のスコア（例：CVaR や標準偏差など）
 - 現時点のプロダクト定義を再利用し、フロントでは単なる数値 + カテゴリ（Low/Med/High）のみ表示
3. **Mean CAS**
 - 採用 Policy の CAS の重み付き平均
 - 重みは Δ¥ またはリーチ（ユーザ数）
 - バックエンド側で計算済の値を受け取る前提
4. **Contribution Chart**
 - `deltaYen` の大きい順に上位 N（5～10）件を抽出
 - 推奨ポートフォリオに含まれていない Policy は表示しない
 - `deltaYen` が負の Policy は棒グラフで下方向に表示してもよい（将来対応）

6. Empty state / エラー文言案

ポリシーが 0 件

タイトル：ポートフォリオを作成するには施策が必要です

本文：

- まだ評価済みのマケ施策がありません。
- 「Policy Lab」で 2 件以上の施策を評価すると、この画面でポートフォリオ最適化が行えます。

ボタン：

- 「Policy Lab を開く」 → `/policy` に遷移
- (デモ環境のみ) 「サンプルポリシーを読み込む」

最適化失敗

「ポートフォリオ最適化が実行できませんでした。
施策数が1件以下、データ欠損、または内部エラーの可能性あります。
施策設定とデータセット構成を確認してください。」

7. 実装タスク一覧 (Claude への依頼用)

1. ドメイン型と API クライアント

- 上記 `PortfolioSummary` / `PortfolioPolicy` / `ParetoPoint` 型を `frontend/src/domain/portfolio.ts` 等に追加。
- `GET /api/portfolio/summary, /policies, /frontier` を叩く関数を `frontend/src/api/portfolio.ts` に実装。

2. React Query フック

- `usePortfolioData(window: PortfolioWindow)` を `frontend/src/hooks/usePortfolioData.ts` に実装。
- 既存の `QueryClient` を利用し、キャッシュキーは `["portfolio-summary", window]` などにする。

3. ページコンポーネント

- `PortfolioPage` を `frontend/src/pages/PortfolioPage.tsx` (または既存ファイル) に実装。
- サイドバーの `/portfolio` ルーティングは現状を再利用。

4. UI コンポーネント

- `PortfolioHeader`
 - `PortfolioSummaryCard`
 - `ParetoFrontierChart`
 - `PortfolioContribution`
 - `PortfolioPoliciesTable`
 - `EmptyPortfolioState`
- を `frontend/src/components/portfolio/` 以下に作成。
既存のカード / テーブルコンポーネント (`shadcn/ui` 等) があればそれを再利用。

5. Styling

- 既存のダークテーマと整合性をとる (背景、カードの角丸、シャドウは他ページに合わせる)。
- Δ¥ は常に円表記を併記 (¥133K のような表記を共通化してフォーマット関数に切り出す)。

6. テスト

- window 切り替え (14/28/56 日) で `QueryKey` が変わり、再フェッチされるかの動作確認。
- ポリシー 0 件の空状態表示。
- API 500/404 時のエラーハンドリング表示。

この仕様をそのまま Claude に渡して、

- 「既存の `/portfolio` ページを、この仕様に沿ってリファクタしてほしい」
- 「既存の API/型を見ながら、差分が最小になるように調整してほしい」

という形で依頼すれば、かなり意図どおりに実装まで持って行きやすいはずです。